

Introducción a Clojure

22 de mayo de 2026

Contenido

- 1 ¿Qué es Clojure?
- 2 Instalación
- 3 Sintaxis Básica
- 4 Estructuras de Datos
- 5 Programación Funcional
- 6 Ejemplos Prácticos
- 7 Herramientas y Ecosistema

Antes de empezar — prueba el código en el navegador

REPL online: sin instalar nada

<https://tryclojure.org/>

Todos los ejemplos de estas diapositivas se pueden probar directamente en el REPL.

`doc` sirve para ver la documentación de una función:

```
(doc map)
(doc filter)
(doc reduce)
```

Ejemplo con `doc`

```
user=> (doc conj)
-----
clojure.core/conj
[coll x] [coll x & xs]
  conj[oin]. Returns a new
  collection with the xs
  added.
```

¿Qué es Clojure?

Definición

Clojure es un dialecto moderno de **Lisp** que corre sobre la **JVM** (Java Virtual Machine). Fue creado por *Rich Hickey* y lanzado en **2007**.

- Lenguaje **funcional** y de **propósito general**
- **Inmutabilidad** por defecto
- Interoperabilidad con **Java** nativa
- REPL interactivo (Read-Eval-Print Loop)
- Concurrencia robusta con STM
- Macros para metaprogramación

Filosofía

- *"Simple made easy"*
- Datos son valores inmutables
- Funciones puras sin efectos
- El código es datos (homoiconicidad)

Plataformas

- **Clojure** → JVM
- **ClojureScript** → JavaScript
- **ClojureCLR** → .NET

Instalación

Guía oficial

La documentación oficial cubre todos los sistemas operativos y se mantiene siempre actualizada:

https://clojure.org/guides/install_clojure

macOS

Homebrew

```
brew install  
clojure/tools/clojure
```

Linux

Script oficial

Requiere `bash`,
`curl`, `rlwrap` y Java

Windows

WSL (recomendado)

o instalador
`clj-msi`

Prerequisito

Todos los métodos requieren **Java** (LTS: 8, 11, 17, 21 o 25). Recomendado: **Adoptium Temurin 25** — <https://adoptium.net/>

Sintaxis — Expresiones S

Todo es una lista: (`operador` `arg1` `arg2` ...)

Clojure usa **notación prefija**. El primer elemento de una lista siempre es el operador o función.

```
; Aritmética básica  
(+ 1 2 3)           ;=> 6  
(* 3 4)             ;=> 12  
(- 10 3)            ;=> 7  
(/ 22 7)            ;=> 22/7  (fracción exacta!)  
(mod 10 3)          ;=> 1
```

```
; Anidamiento natural  
(+ (* 2 3) (- 10 4)) ;=> 12
```

```
; Comparaciones  
(= 1 1)             ;=> true  
(> 5 3)             ;=> true  
(not= 1 2)          ;=> true
```

Sintaxis — Variables y Funciones

Definir valores con `def`

```
(def pi 3.14159)
(def nombre "Closure")
(def activo? true)

; Usar el valor
(println nombre)
;=> Closure
```

Función anónima con `fn`

```
; fn crea una función
(fn [x] (* x x))

; Llamar directamente
((fn [x] (* x x)) 5)
;=> 25
```

Definir funciones con `defn`

```
(defn cuadrado
  "Eleva x al cuadrado"
  [x]
  (* x x))
(cuadrado 7) ;=> 49

; Múltiples argumentos
(defn suma [a b] (+ a b))
(suma 3 4) ;=> 7

; Aridad múltiple
(defn saludo
  ([ ] ";Hola!")
  ([nombre] (str ";Hola, " nombre "!")))
(saludo) ;=> ";Hola!"
(saludo "Ana") ;=> ";Hola, Ana!"
```

Sintaxis — Control de Flujo

if y when

```
(if (> 5 3)
  "cinco es mayor"
  "tres es mayor")    ;=> "cinco es
                        mayor"

(when (pos? 5)
  (println "positivo")) ;=>
  positivo
```

cond — múltiples condiciones

```
(defn clasificar [n]
  (cond
    (neg? n)  "negativo"
    (zero? n) "cero"
    :else     "positivo"))
```

let — variables locales

```
(let [x 10
      y 20
      suma (+ x y)]
  (println "Suma:" suma)
  suma)
;=> Suma: 30
```

Recursión con loop/recur

```
(defn factorial [n]
  (loop [acc 1, i n]
    (if (= i 0)
      acc
      (recur (* acc i) (dec i)))))

(factorial 5)    ;=> 120
```


Estructuras de Datos — Listas y Vectores

[Lista] Listas (...)

Estructura secuencial. Inserción eficiente al frente.

```
(def nums '(1 2 3)) ; quoted list

; Operaciones básicas
(first nums)          ;=> 1
(rest  nums)          ;=> (2 3)
(count  nums)         ;=> 3
(cons 0 nums)         ;=> (0 1 2 3)
(conj  nums 0)        ;=> (0 1 2 3)
```

[Vector] Vectores [...]

Acceso **aleatorio** eficiente por índice.

```
(def v [10 20 30 40])

; Acceso
(first v)          ;=> 10
(last  v)          ;=> 40
(nth   v 2)        ;=> 30
(count v)          ;=> 4

; Agregar / actualizar
(conj v 50)        ;=> [10 20 30 40 50]
(assoc v 1 99)     ;=> [10 99 30 40]
```

[Mapa] Mapas {...}

Colecciones clave-valor (*hash maps*).

```
(def persona {:nombre "Ana" :edad 28})  
(get persona :nombre)    ;=> "Ana"  
(:edad persona)          ;=> 28  
(assoc persona :edad 29) ; nuevo  
                           mapa  
(dissoc persona :edad)   ; sin  
                           clave  
(keys persona)           ;=> (:nombre :edad)  
(vals persona)           ;=> ("Ana" 28)
```

[Conj] Conjuntos #{...}

Colección sin duplicados.

```
(def colores #{"rojo" "verde" "azul"  
              "})  
(contains? colores "rojo") ;=> true  
(colores "verde")          ;=> "  
    verde"  
(colores "blanco")         ;=> nil  
(conj colores "amarillo")  
;=> #{"rojo" "verde" "azul" "  
    amarillo"}  
(disj colores "rojo")  
;=> #{"verde" "azul" }
```

Tipo	Sintaxis	Acceso	Orden	Duplicados
Lista	'(1 2 3)	$O(n)$ secuencial	Sí	Sí
Vector	[1 2 3]	$O(1)$ por índice	Sí	Sí
Mapa	{:k v}	$O(1)$ por clave	No	No (claves)
Conjunto	#{1 2 3}	$O(1)$ pertenencia	No	No

[!] Inmutabilidad

Todas las estructuras en Clojure son **persistentes e inmutables**. Las "modificaciones" retornan una **nueva estructura** que comparte memoria con la original (*structural sharing*).

Funciones de Alto Orden

map — transformar colecciones

```
(map inc [1 2 3 4])  
=> (2 3 4 5)
```

```
(map #(* % 2) [1 2 3])  
=> (2 4 6)
```

```
(map str [1 2 3])  
=> ("1" "2" "3")
```

reduce — acumular

```
(reduce + [1 2 3 4 5])      => 15  
(reduce * 1 [1 2 3 4 5])    => 120
```

```
; Construir un mapa  
(reduce #(assoc %1 (first %2) (  
    second %2))  
    {} [[:a 1] [:b 2]])  
=> {:a 1, :b 2}
```

filter — filtrar elementos

```
(filter even? [1 2 3 4 5 6])  
=> (2 4 6)
```

```
(filter #(> % 3) [1 2 3 4 5])  
=> (4 5)
```

comp y partial

```
(def doble-inc (comp inc #(* 2 %)))  
(doble-inc 3)      => 7
```

```
(def suma5 (partial + 5))  
(suma5 10)      => 15
```

El Operador Thread $\rightarrow$ y $\rightarrow\!\!\gg$

Encadenamiento legible de transformaciones

Evita el anidamiento profundo; pasa el resultado de izquierda a derecha.

Sin threading (difícil de leer)

```
; Calcular suma de cuadrados  
; de números pares del 1 al 10  
(reduce +  
  (map #(* % %)  
    (filter even?  
      (range 1 11))))  
;=> 220
```

Con $\rightarrow\!\!\gg$ (mucho más claro)

```
(->> (range 1 11)  
      (filter even?)  
      (map #(* % %))  
      (reduce +))  
;=> 220  
  
; -> inserta como 1er argumento  
(-> " hola mundo "  
    clojure.string/trim  
    clojure.string/upper-case)  
;=> "HOLA MUNDO"
```

Ejemplo 1 — FizzBuzz

Objetivo: clasificar números según si son múltiplos de 3 (Fizz), de 5 (Buzz) o de ambos (FizzBuzz), y generar la secuencia completa.

```
(defn fizzbuzz [n]
  (cond
    (zero? (mod n 15)) "FizzBuzz"
    (zero? (mod n 3))  "Fizz"
    (zero? (mod n 5))  "Buzz"
    :else              (str n)))

; Generar FizzBuzz del 1 al 20
(doseq [n (range 1 21)]
  (println (fizzbuzz n)))

; Versión funcional con map
(map fizzbuzz (range 1 21))
;=> ("1" "2" "Fizz" "4" "Buzz" "Fizz" "7" "8" "Fizz" "Buzz"
     "11" "Fizz" "13" "14" "FizzBuzz" "16" "17" "Fizz" "19" "Buzz")
```

Ejemplo 2 — Manejo de Datos

Objetivo: trabajar con una colección de mapas para filtrar, calcular promedios y ordenar resultados.

```
(def estudiantes
  [{:nombre "Ana"      :nota 85} {:nombre "Luis"      :nota 92}
   {:nombre "María"    :nota 78} {:nombre "Carlos"    :nota 95}
   {:nombre "Sofía"    :nota 61}])

; Aprobados (nota >= 70)
(def aprobados (filter #(>= (:nota %) 70) estudiantes))

; Promedio
(def promedio
  (/ (reduce + (map :nota estudiantes)) (count estudiantes)))

; Top 3 por nota
(def top3
  (->> estudiantes (sort-by :nota >) (take 3) (map :nombre)))
;=> ("Carlos" "Luis" "Ana")
```

Ejemplo 3 — Fibonacci y Secuencias Lazy

Objetivo: comparar recursión tradicional con una secuencia infinita evaluada solo cuando se necesita.

```
; Fibonacci recursivo
(defn fib [n]
  (if (< n 2) n (+ (fib (- n 1)) (fib (- n 2)))))

; Secuencia INFINITA lazy
(def fibonacci
  (lazy-seq
    (letfn [(gen [a b]
              (cons a (lazy-seq (gen b (+ a b)))))
            (gen 0 1)])))

; Tomar solo los que necesitamos
(take 10 fibonacci)
;=> (0 1 1 2 3 5 8 13 21 34)

; Fibonacci pares
(->> fibonacci (filter even?) (take 5))
;=> (0 2 8 34 144)
```


Ejemplo 4 — Interop con Java

Objetivo: mostrar cómo Clojure usa clases, métodos y objetos Java directamente desde el código.

```
(import java.time.LocalDate)

(def hoy (LocalDate/now))      ; 2025-01-15
(.getYear      hoy)           ;=> 2025
(.getDayOfWeek hoy)           ;=> WEDNESDAY

; Métodos estáticos
(Math/sqrt 144.0)              ;=> 12.0
(Math/pow 2 10)                ;=> 1024.0

; Strings
(.toUpperCase "clojure")      ;=> "CLOJURE"

; doto: llamadas encadenadas sobre un objeto
(doto (java.util.ArrayList.)
  (.add 1) (.add 2) (.add 3))
;=> [1 2 3]
```

Herramientas del Ecosistema

[Tools] Gestores de proyectos

- **Leiningen** — el más popular; maduro y con plugins
- **deps.edn** — oficial de Clojure CLI; minimalista
- **Boot** — alternativa basada en pipelines

[IDE] Editores recomendados

- **VS Code** + Calva
- **Emacs** + CIDER
- **IntelliJ** + Cursive
- **Neovim** + Conjure

[Libs] Librerías populares

- **Ring** — HTTP servidor web
- **Compojure** — routing web
- **Reagent** — React en ClojureScript
- **core.async** — programación asíncrona
- **Datomic** — base de datos funcional
- **clojure.spec** — especificación de datos
- **Pedestal** — framework web robusto

¿Por dónde seguir?

[Libros]

Recursos

clojure.org
clojuredocs.org
4clojure.oxal.org
exercism.io/clojure

[Estudio]

Libros

Clojure for the Brave
Programming Clojure
Getting Clojure
Web Dev with Clojure

[Comunidad]

Comunidad

Clojurians Slack
r/Clojure
ClojureVerse
Clojure/conj (conf.)

"Simple is not easy, but it's worth it."

— Rich Hickey